# Surihana Vows Client Clone and Deployment Audit

Source-of-truth operating guide for cloning this template, curating it for a new wedding client, provisioning infrastructure, migrating the database cleanly, deploying on Vercel, and handing the finished platform over without cross-client leakage.

| | |
|---|---|
| **Prepared from repo state** | 2026-03-20 |
| **Workspace** | `d:\Surihana Vows\Surihana Projects\surihana-vows-template` |
| **Stack** | Git / GitHub, Next.js 14, Vercel, Supabase, custom domain, Resend |
| **Primary intent** | Turn this template into a repeatable client-delivery system |

> **Recommended operating model:** one Git repo, one Vercel project, one Supabase project, one email/domain setup, and one production domain per client. Do not serve multiple client weddings from the same Supabase project or production deployment.

---

## 1. Executive Summary

This codebase is a workable client template, but the safest cloning process is more manual than the existing docs suggest. The front-end wedding content is primarily driven by local JSON config files in `config/`, while operational features such as guests, RSVP, uploads, family vault access, predictions, and planning modules depend on Supabase tables and storage buckets.

For clean delivery, treat the repo as the presentation layer and content seed, and treat Supabase as the per-client data store. Provision a fresh Supabase project for each client, run the schema once on that blank project, create client-specific accounts, import the guest list, verify the invite flow, then connect the final domain and set `NEXT_PUBLIC_SITE_URL` to that exact live URL.

> **Important audit conclusion:** the existing `DEPLOY.md` and some setup scripts are partially out of sync with the codebase. This document should be treated as the operational source of truth until those files are refreshed.

## 2. What Lives Where

| Layer | What it controls | Where it lives | Client-clone rule |
|---|---|---|---|
| Code and UI | Routes, components, invite experience, dashboards, API routes | Git repo / Vercel deployment | Duplicate per client or branch from a locked template baseline |
| Static wedding content | Names, copy, schedule, travel guide, gallery defaults, theme labels | `config/*.json` | Always rewrite for each client |
| Theme implementation | Color tokens, visual identity, typography direction | `app/globals.css`, `styles/globals.css`, `themes/*` | Check before delivery; theme switching is not fully config-only |
| Runtime data | Guests, RSVP, uploads, guestbook, family vault, predictions, vendors, polls | Supabase Postgres | Use a fresh project per client |
| Media storage | Photos, videos, private family vault assets | Supabase Storage buckets | Use a fresh storage set per client |
| Secrets and hosting | Environment variables, runtime keys, production URL | Vercel project settings | Never reuse another client's env set |
| Email and domain | Magic links, sender identity, live invite URLs | Resend + DNS + Vercel | Verify domain before go-live |

### Static configuration files that must be curated

| File | Purpose | Typical client edits |
|---|---|---|
| `config/wedding.json` | Main wedding identity and invitation copy | Names, date, times, venues, map links, story, rituals, thank-you copy, dress code |
| `config/events.json` | Events page schedule | Add ceremony, reception, sangeet, mehendi, brunch, after-party, etc. |

| File | Purpose | Typical client edits |
|---|---|---|
| `config/travel.json` | Travel page content | Airport, hotels, transfers, FAQs, local tips, essentials |
| `config/gallery.json` | Gallery categories and slideshow defaults | Replace demo imagery, captions, post-wedding download link |
| `config/predictions.json` | Prediction questions and reveal timing | Rewrite questions and set the real reveal timestamp |
| `config/theme.json` | Brand labels and top-level navigation labels | Site name, tagline, nav wording, stage copy |

### Assets that are outside JSON

- `public/audio/wedding-theme.mp3` controls the invite background music.
- Theme CSS is selected in `app/globals.css`; available theme folders are `warm`, `minimal`, and `jewel`.
- `next.config.js` already allows remote images from Unsplash and any `*.supabase.co` storage path.

## 3. Runtime Data and Auth Audit

The app does not rely only on static config. Several user and guest features require a real database. The current migration runner expects the following public tables after setup:

| Table group | Tables |
|---|---|
| Core guest experience | `weddings`, `guests`, `rsvp`, `events`, `travel_info`, `invite_analytics` |
| Live and media | `guest_messages`, `photos`, `videos` |
| Private access and family area | `family_users`, `family_posts`, `family_magic_links`, `invite_access_codes` |
| Lifecycle and long-term archive | `wedding_stage_overrides`, `time_capsules`, `guest_predictions` |
| Planning and collaboration | `vendors`, `family_polls`, `family_poll_votes` |

Storage is also part of the platform contract. The current setup expects four buckets: `couple-photos`, `guest-uploads`, `wedding-videos`, and `family-vault`.

> **Auth model:** family and admin access is custom. This repo stores users in `family_users` with PBKDF2 password hashes and also supports magic links through `family_magic_links`. Do not plan your client deployment around Supabase Auth users.

## Environment variables by purpose

| Variable | Used for | Required when |
|---|---|---|
| `NEXT_PUBLIC_SUPABASE_URL` | Supabase URL for browser, server, scripts | Required for any real client environment |
| `NEXT_PUBLIC_SUPABASE_ANON_KEY` | Public browser/server Supabase client | Required for real runtime |
| `SUPABASE_SERVICE_ROLE_KEY` | Admin APIs, private reads, scripts, storage setup | Required for full runtime and setup |
| `SUPABASE_ACCESS_TOKEN` | Supabase Management API database migration runner | Required only while running `scripts/setup-database.ts` |
| `NEXT_PUBLIC_SITE_URL` | Invite links, general invite URL, email links | Must match the final live domain |
| `AUTH_SECRET` | Session signing | Required in production |
| `RESEND_API_KEY` | Magic link and email delivery | Required if emails must send in production |
| `EMAIL_FROM` | Sender address for outgoing email | Required with Resend |
| `FAMILY_LOGIN_EMAIL`, `FAMILY_LOGIN_PASSWORD`, `ADMIN_LOGIN_EMAIL`, `ADMIN_LOGIN_PASSWORD` | Demo or fallback credentials | Useful locally, not a substitute for real user creation |
| `NEXT_PUBLIC_LIVESTREAM_URL` | Wedding day live stream | Optional |

# 4. Recommended Client Cloning Workflow

### Phase A: create a clean client container

1. Create a new private GitHub repo from the template baseline. A separate repo per client is cleaner than keeping multiple clients in one repo.
2. Clone that new repo locally.
3. Create a dedicated Vercel project for the client repo.
4. Create a brand-new Supabase project for the client. Never point two clients to the same database or storage.
5. Create or assign the final client domain and decide where DNS will be managed.

```
git clone https://github.com/your-org/client-wedding-repo.git
cd client-wedding-repo
npm install
```

### Phase B: provision secrets and local environment

1. Copy `.env.example` to `.env.local`.
2. Paste the new client's Supabase URL, anon key, service role key, and personal access token.
3. Generate a new `AUTH_SECRET`.
4. Set `NEXT_PUBLIC_SITE_URL` to the final production domain if known, otherwise set the preview domain temporarily and update it before go-live.
5. Configure Resend values if the deployment must send emails.

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

### Phase C: migrate database on the blank project

The current migration runner reads every `.sql` file inside `database/` in lexicographic order and sends it to the Supabase Management API.

```
node --experimental-strip-types scripts/setup-database.ts
node --experimental-strip-types scripts/setup-storage.ts
```

> **Safe rule:** run the full database pack once on a fresh Supabase project. Do not assume it is safe to rerun the entire folder against an existing production project, because `phase-2-supabase.sql` contains unguarded `CREATE POLICY` statements and the folder also includes repair-style SQL such as `fix_family_users.sql`.

For each new client project, the clean migration practice is:

1. Start with a new empty Supabase project.
2. Run `scripts/setup-database.ts` once.
3. Run `scripts/setup-storage.ts` once as a validation and bucket repair step.
4. If you later need to upgrade an existing client project, review and apply only the required SQL, not the whole folder blindly.
5. Before any later production schema change, export a database backup or snapshot.

### Phase D: curate the client content and theme

1. Rewrite all six JSON config files in `config/`.
2. Replace the background music in `public/audio/wedding-theme.mp3`.
3. Choose the visual theme in `app/globals.css`. The current build imports the `warm` theme by default.
4. Check `config/theme.json` for brand labels and navigation labels.
5. If the couple wants a strong visual departure, review theme tokens in `themes/warm`, `themes/minimal`, or `themes/jewel`.

> **Practical theme rule:** most copy changes are JSON-only, but the actual color system is still selected by CSS import. Treat theme choice as a code-level presentation setting, not a pure CMS field.

### Phase E: create users and guest records

Create admin or family login users with the repo script, not via Supabase Auth. Import guests from CSV so every guest gets a unique invite code.

```
node --experimental-strip-types scripts/create-user.ts --email admin@clientdomain.com --pass
node --experimental-strip-types scripts/create-user.ts --email family@clientdomain.com --pas
node --experimental-strip-types scripts/import-guests.ts --file guests.csv --weddingId YOUR_
```

The guest CSV parser expects at least a `guestName` or `name` column, and can also use `familyName` and `phone`.

### Phase F: deploy and connect the domain

1. Link the repo to Vercel.
2. Add all runtime environment variables in the Vercel project settings.
3. Deploy a preview and test flows before production.
4. Add the custom domain in Vercel and follow the DNS records Vercel provides.
5. After the custom domain resolves, update `NEXT_PUBLIC_SITE_URL` to that exact domain and redeploy.

```
vercel
vercel --prod
```

---

## 5. How to Change Client Details, Plans, and Theme

### Client details

- Names, dates, venues, story, dress code, rituals, and thank-you copy live in `config/wedding.json`.
- Program order and multi-event details live in `config/events.json`.
- Travel details, FAQs, emergency info, and arrival tips live in `config/travel.json`.
- Prediction questions and reveal timing live in `config/predictions.json`.
- Gallery categories and hero images live in `config/gallery.json`.

### Plans and operational modules

- If by "plans" you mean the wedding platform stages, the stage copy is split between `config/theme.json` and `config/wedding.json`.
- If you mean day-of planning modules such as vendors, polls, and vault content, those are database-driven and should be populated after deployment from the admin side or direct data entry.
- For a clean clone, start those planning tables empty for each client. Do not copy another wedding's live data.

### Theme

- The app currently imports `themes/warm/theme.css` from `app/globals.css`.
- Alternative theme folders currently available are `minimal` and `jewel`.

- If you want a different default theme for a client, switch the import and verify the visual result on all pages before go-live.
- The color tokens also have fallbacks in `styles/globals.css`, so theme changes should be tested instead of assumed.

---

## 6. Database Migration SOP

The clean database story for this template is: new client equals new Supabase project. This avoids data overlap, avoids trying to surgically clean old rows, and turns the migration process into a deterministic one-time setup on a blank database.

### Clean migration checklist

1. Create a new Supabase project in the region closest to the client's main audience.
2. Set `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, and `SUPABASE_ACCESS_TOKEN` in `.env.local`.
3. Run `scripts/setup-database.ts` once.
4. Run `scripts/setup-storage.ts` once.
5. Verify the four buckets and expected tables.
6. Create admin/family users.
7. Import the guest list and distribute invite links only after QA.

### What not to do

- Do not reuse a previous client's Supabase project and manually delete rows.
- Do not assume the SQL folder is a perfect idempotent migration history.
- Do not store temporary one-off repair SQL in `database/` unless you intend future fresh projects to execute it.
- Do not treat the old deployment doc table counts or file counts as authoritative.

---

## 7. QA Before Delivery

Before the client sees the production domain, test the following on the preview deployment and then again on production:

- Home page and invitation page load correctly on desktop and mobile.
- A personalized guest invite link resolves correctly.

- The invite opens and the site navigation works across invitation, story, events, travel, guestbook, gallery, and predictions.
- RSVP submit path works.
- Guestbook submission works.
- Predictions submit and reveal timing behave correctly.
- Photo upload and gallery reads work if enabled.
- Admin login works.
- Family login or magic link flow works.
- Email delivery works from the real sender domain.
- `NEXT_PUBLIC_SITE_URL` is the live domain, not a preview URL or localhost value.

---

## 8. Delivery and Handover Package

For a clean client delivery, hand over or internally archive the following:

- GitHub repo URL for that client project
- Vercel project URL and owner/team details
- Supabase project reference, region, and ownership details
- Domain registrar and DNS access owner
- Resend account or sender domain ownership details
- Final environment variable inventory stored in a password manager
- Admin login email and password, or the process to rotate them
- Client-facing operating notes: how to access admin, how to approve uploads, how to view messages, how to add vault content

---

## 9. Audit Findings and Risks in the Current Repo

| Finding | Why it matters | Operational rule for now |
|---|---|---|
| `DEPLOY.md` is outdated | It still references old table counts, fewer SQL files, and an older setup story | Use this handbook as the cloning source of truth |
| `scripts/create-wedding.ts` is | It ignores the promise of generic CLI scaffolding and does not currently write the | Do not rely on it as a generic client scaffold; |

| Finding | Why it matters | Operational rule for now |
|---|---|---|
| hardcoded to one wedding | `weddings` row it claims to insert | curate config files manually unless the script is repaired later |
| `scripts/create-wedding.ts` does not write every config file | The repo uses six config files, but the script does not generate `predictions.json` | Always review all six config files by hand |
| `scripts/setup-database.ts` comments are stale | The script actually needs `NEXT_PUBLIC_SUPABASE_URL` and `SUPABASE_ACCESS_TOKEN`; the comment still mentions `SUPABASE_SERVICE_ROLE_KEY` | Trust the code path, not the header comment |
| `scripts/setup-database.ts` runs every SQL file in `database/` | Repair files and support SQL are mixed into the migration folder and will run for future fresh projects too | Keep the folder clean and review new SQL before using it for another client |
| `phase-2-supabase.sql` is not fully rerunnable | It contains plain `CREATE POLICY` statements without guards, despite comments saying it is safe to rerun | Run the full SQL pack only on blank projects; review manually for later upgrades |
| `verify-deploy.ps1` is useful but incomplete | It still verifies an older subset of tables and should not be the only sign-off criterion | Use it as one check, then do manual QA on the real flows |
| Theme selection is partly code-level | Theme changes are not controlled only by JSON; CSS imports still decide the active theme | Test visual changes on the actual pages before delivery |

## 10. Recommended Internal Improvement Backlog

This section is not required to deliver the next client, but it would reduce risk and delivery time on future clones:

- Repair `scripts/create-wedding.ts` so it truly accepts CLI input, writes all config files, and optionally inserts the wedding row.
- Separate baseline migrations from repair SQL and support SQL.
- Make `phase-2-supabase.sql` genuinely idempotent.

- Refresh `DEPLOY.md` and `verify-deploy.ps1` to match the current schema.
- Make theme selection fully config-driven if rapid client branding is a core business need.

---

## 11. Final Recommended Delivery Sequence

1. Fork or duplicate the template into a client-specific private repo.
2. Create a fresh Supabase project and a fresh Vercel project.
3. Set client-specific environment variables.
4. Run database and storage setup on the blank Supabase project.
5. Rewrite all six config files and replace assets.
6. Create admin and family users.
7. Import guests.
8. Deploy preview and complete full QA.
9. Connect the production domain and update `NEXT_PUBLIC_SITE_URL`.
10. Deploy production, test again, then deliver.

> **Bottom line:** this template is deliverable for client work if you clone it with fresh infrastructure per wedding, treat `config/*.json` as the copy source of truth, treat Supabase as the per-client data layer, and avoid relying on the older deployment docs or the current hardcoded wedding scaffold script.